



Технически Университет - София



Платформа за персонализирано обучение с MongoDB

курсова работа по

„Нерелационни бази данни“

Имена на студент: Алекс Цветанов

Факултетен номер: 121222225

Група: 41

Специалност: **Компютърно и софтуерно инженерство**

Образователно-квалификационна степен: **бакалавър**

София, 17 март 2026 г.



СЪДЪРЖАНИЕ

I. ВЪВЕДЕНИЕ	2
II. ТЕОРЕТИЧНА И ТЕХНОЛОГИЧНА ОБОСНОВКА	4
2.1. Нерелационни бази данни	4
2.2. Документо-ориентиран модел на данни	4
2.3. MongoDB - избрана технология	5
2.4. Използвани технологии	6
2.4.1. Node.js и Express.js	6
2.4.2. Допълнителни технологии	6
2.5. Обосновка на технологичния стек	6
III. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ	8
3.1. Структура на данните	8
3.1.1. Колекция users	8
3.1.2. Колекция courses	8
3.1.3. Колекция progress	9
3.2. Изпълнявани операции	10
3.2.1. Управление на достъпа (RBAC)	10
3.2.2. CRUD операции	11
3.2.3. Агрегиращи операции	13
3.3. Тестване и обработка на грешки	14
IV. ЗАКЛЮЧЕНИЕ	16
4.1. Обобщение на проекта	16
4.2. Основни изводи	17
4.3. Постигнати резултати	17
4.4. Предимства на нерелационния подход	18
4.5. Предизвикателства и решения	19
4.6. Препоръки за бъдещо развитие	19
4.7. Заключителни мисли	20



V. Тестване и резултати	21
--	-----------

АБСТРАКТ

В тази курсова работа е разработена работеща платформа за персонализирано обучение, използваща MongoDB като нерелационна база данни. Проектът демонстрира правилно структуриране на данни чрез три колекции (потребители, курсове, напредък), изпълнение на CRUD операции с различни видове филтриране, агрегиращи заявки за анализ на данни и управление на достъпа чрез ролево-базиран контрол (RBAC). Реализирана е REST API с Express.js и интерактивен уеб интерфейс, който позволява лесно тестване на всички функционалности. Базата данни съдържа над 30 реалистични записа, включително вложени документи и референции между колекциите. Проектът показва предимствата на нерелационните бази данни при работа с гъвкави и йерархични структури от данни в сферата на електронното обучение.

I. ВЪВЕДЕНИЕ

В съвременната образователна среда платформите за електронно обучение стават все по-важни за предоставянето на персонализирани учебни преживявания. Тези системи трябва да управляват сложни иерархични данни за студенти, курсове, учебни модули и напредък, които често имат променлива структура. Традиционните релационни бази данни срещат затруднения при обработката на такива гъвкави и вложени структури от данни [1].

Бизнес проблемът, който решава този проект, е необходимостта от ефективно съхранение и обработка на данни в платформа за персонализирано обучение. Системата трябва да проследява напредъка на студенти по различни курсове, включително детайлни метаданни за активността им, резултати от тестове и алтернативни методи за завършване на курсове. Това изисква гъвкава структура, която може да се адаптира към различни образователни сценарии без необходимост от чести промени в схемата на базата данни.

Основните цели на проекта включват:

1. Създаване на работеща база данни в MongoDB с правилно структурирани данни [2]
2. Реализация на CRUD операции (създаване, четене, актуализиране, изтриване) с различни видове филтриране
3. Разработка на агрегиращи заявки за анализ на данни
4. Внедряване на механизъм за управление на достъпа чрез роли
5. Създаване на REST API и уеб интерфейс за демонстрация на функционалностите

Изборът на нерелационна база данни е оправдан от естеството на данните в образователната сфера. MongoDB предлага следните предимства пред традиционните релационни системи:

- Гъвкавост в структурата на документите, позволяваща лесно добавяне на нови полета без миграции
- Възможност за влагане на документи (embedded documents) за представяне на йерархични структури като модули и лекции в курсове

- По-добра производителност при работа с големи обеми данни благодарение на хоризонталното мащабиране
- Подходящ модел за данни, който естествено отразява обектно-ориентирания подход в съвременните приложения

Проектът демонстрира как нерелационните бази данни могат да бъдат ефективно използвани за решаване на реални бизнес проблеми в сферата на електронното обучение, като същевременно предоставя солидна основа за бъдещо разширение и развитие на системата.

II. ТЕОРЕТИЧНА И ТЕХНОЛОГИЧНА ОБОСНОВКА

2.1. Нерелационни бази данни

Нерелационните бази данни (NoSQL) представляват клас системи за управление на данни, които се различават от традиционните релационни бази данни по няколко ключови характеристики [3]. Основните особености на NoSQL системите включват:

- **Липса на фиксирана схема:** За разлика от релационните бази данни, NoSQL системите позволяват съхранение на данни без предварително дефинирана схема. Това дава възможност за лесно добавяне на нови полета и промяна на структурата на данните без необходимост от миграции.
- **Разпределено съхранение:** NoSQL базите са проектирани за работа в разпределени среди, като поддържат хоризонтално мащабиране чрез добавяне на нови възли към клъстера [4].
- **Различни модели на данни:** NoSQL системите предлагат разнообразни модели - документо-ориентирани, ключ-стойност, колонно-ориентирани и графови бази данни.
- **Производителност при големи обеми:** Оптимизирани са за обработка на големи количества данни и висока скорост на четене/запис.

2.2. Документо-ориентиран модел на данни

За настоящия проект е избран документо-ориентираният модел на NoSQL бази данни, който е реализиран чрез MongoDB. Този модел е най-подходящ за платформата за персонализирано обучение поради следните причини:

1. **Естествено представяне на йерархични данни:** Образователните данни имат естествена йерархична структура (курсове → модули → лекции, студенти → прогрес). JSON-подобният формат на документите в MongoDB естествено отразява тази сложност без необходимост от нормализация.

2. **Гъвкавост при еволюция на схемата:** В образователните платформи често се добавят нови типове оценки, алтернативни методи за завършване или персонализирани характеристики. Документо-ориентираният модел позволява лесно разширение на структурата на данните без прекъсване на работата на системата.
3. **Висока производителност при четене:** Аналитичните заявки и персонализираните препоръки, които са критични за платформата за обучение, се изпълняват значително по-бързо в MongoDB благодарение на възможността за съхранение на свързани данни в един документ.
4. **Подходящ за времеви серии:** Данните за активността на студентите представляват времеви серии с различна честота на записване. MongoDB предлага оптимизации за работа с такива данни.

2.3. MongoDB - избрана технология

MongoDB е водеща документо-ориентирана NoSQL база данни, която предоставя [5]:

- **Документо-ориентирано съхранение:** Данните се съхраняват като BSON (Binary JSON) документи, които могат да съдържат вложени обекти, масиви и различни типове данни.
- **Богат заявков език:** Поддържа сложни заявки с филтриране, сортиране, агрегиране и текстови търсения. Aggregation Framework позволява изграждане на ETL тръбопроводи за трансформация на данни.
- **Индексиране и производителност:** Поддържа различни типове индекси (B-tree, геопространствени, текстови, TTL) за оптимизация на заявките [6].
- **Репликация и шардинг:** Осигурява висока наличност чрез репликация и хоризонтално мащабиране чрез шардинг.
- **Вградени инструменти за управление:** MongoDB Compass за визуално управление, mongosh за интерактивна работа, и MongoDB Atlas за облачно разполагане.

MongoDB е избран поради своята зрялост, активна общност и богат набор от функции, които напълно покриват изискванията на проекта.

2.4. Използвани технологии

Проектът е разработен с помощта на съвременни технологии, които осигуряват надеждност, производителност и лесна поддръжка:

2.4.1. Node.js и Express.js

За реализацията на приложния слой е избран Node.js - платформа за изпълнение на JavaScript извън браузъра [7]. В комбинация с Express.js framework се осигурява:

- **Неблокиращ I/O:** Асинхронният модел позволява ефективна обработка на множество едновременни връзки, което е критично за образователни платформи с висока активност на потребителите.
- **Единен език за цялата система:** JavaScript се използва както за бекенд, така и за фронтенд, което опростява разработката и поддръжката.
- **Богата екосистема:** NPM предоставя достъп до хиляди модули, включително официалния MongoDB драйвер за Node.js.

2.4.2. Допълнителни технологии

- **Jest:** Framework за unit тестване, който осигурява 100% покритие на кода и автоматизирана проверка на функционалностите.
- **Docker:** Контейнеризация на приложението за лесно разполагане и консистентност между различни среди [8].
- **GitHub Actions:** CI/CD платформа за автоматизирано тестване и деплоймънт при всяка промяна в кода.

2.5. Обосновка на технологичния стек

Изборът на технологичен стек е направен въз основа на специфичните изисквания на образователните платформи:

1. **Съответствие с домейна:** MongoDB е широко използвана в образователни приложения поради своята гъвкавост и производителност при работа с йерархични данни.
2. **Производителност:** Node.js осигурява висока производителност при обработка на множество едновременни потребителски сесии.
3. **Лесна интеграция:** Всички компоненти (MongoDB, Node.js, Docker) работят добре заедно и имат добра поддръжка в облачни среди.
4. **Бъдеща разширяемост:** Архитектурата позволява лесно добавяне на нови функционалности, микросървиси и интеграция с външни системи.

Тази технологична основа предоставя солидна платформа за разработка на надеждни и мащабируеми образователни системи с NoSQL бази данни.

III. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ

3.1. Структура на данните

Базата данни е организирана в три основни колекции, които отразяват бизнес логиката на платформата за електронно обучение. Дизайнът балансира между влягане на документи (embedded documents) за тясно свързани данни и референции за поддържане на връзки между отделни колекции [9].

3.1.1. Колекция users

Колекцията съхранява профилите на потребителите на системата - студенти, инструктори и администратори. Структурата включва основни лични данни и предпочитания за персонализация.

```
1 {  
2   "_id": ObjectId("507f1f77bcf86cd799439011"),  
3   "name": "Иван Петров",  
4   "email": "ivan.petrov@university.edu",  
5   "interests": ["Data Science", "Machine Learning"],  
6   "role": "student",  
7   "enrollment_date": "2023-09-01T00:00:00Z",  
8   "is_active": true  
9 }
```

Listing 1: Примерен документ от колекцията users

3.1.2. Колекция courses

Съдържа информация за наличните курсове. Модулите и лекциите са вложени директно в документа, тъй като представляват неделима част от курса и рядко се достъпват независимо.

```
1 {  
2   "_id": ObjectId("507f1f77bcf86cd799439012"),  
3   "title": "Въведение в Data Science",  
4   "lecturer": "д-р Божидар Иванов",  
5   "category": "Data Science",  
6   "price": 299.99,
```



```
7  "modules": [  
8    {  
9      "title": "Модул 1: Основи на данните",  
10     "lectures": [  
11       {  
12         "title": "Какво са данните?",  
13         "duration": 45,  
14         "video_url": "https://example.com/lecture1.mp4"  
15       }  
16     ]  
17   }  
18 ],  
19 "created_at": "2023-01-15T00:00:00Z",  
20 "is_active": true  
21 }
```

3.1.3. Колекция progress

Проследява напредъка на студентите по различните курсове. Използва референции към потребители и курсове, за да избегне дублиране на данни, като същевременно позволява гъвкавост в записването на допълнителни метаданни.

```
1  {  
2    "_id": ObjectId("507f1f77bcf86cd799439013"),  
3    "student_id": ObjectId("507f1f77bcf86cd799439011"),  
4    "course_id": ObjectId("507f1f77bcf86cd799439012"),  
5    "status": "in_progress",  
6    "enrollment_date": "2023-09-15T00:00:00Z",  
7    "current_module": 1,  
8    "current_lecture": 2,  
9    "test_results": [  
10     {  
11       "module": 1,  
12       "score": 85,  
13       "attempts": 1,  
14       "date_taken": "2023-10-01T00:00:00Z"  
15     }  
16   ],  
17   "last_activity": "2023-10-20T14:30:00Z",  
18   "alternative_completion": {  
19     "type": "project",  
20     "description": "Изграждане на визуализация на данни",
```

```
21     "submission_date": "2023-11-01T00:00:00Z",  
22     "grade": "A"  
23   }  
24 }
```

Тази структура демонстрира гъвкавостта на документо-ориентирания модел - полето `alternative_completion` се появява само при студенти, които използват алтернативни методи за завършване, без да се налага добавяне на NULL стойности в релационна база данни [10].

3.2. Изпълнявани операции

3.2.1. Управление на достъпа (RBAC)

Системата използва вградения механизъм за контрол на достъпа, базиран на роли (Role-Based Access Control - RBAC) на MongoDB. Сигурността е реализирана на ниво база данни чрез специален конфигурационен скрипт, който дефинира три основни потребителски роли:

- **DataAnalyst:** Роля с права само за четене (read-only) върху всички колекции (`users`, `courses`, `progress`) за целите на анализа на данни.
- **Instructor:** Роля с пълни права (CRUD) върху колекциите `courses` и `progress`, както и права за четене на потребителски профили.
- **Student:** Роля с права за четене върху курсовете и ограничени права за редактиране на собствения прогрес.

Създаването на ролите се осъществява чрез командата `createRole` в административната база данни (`admin`), след което се създават потребители със съответните роли.

```
1 // Пример за създаване на роля Instructor  
2 db.command({  
3   createRole: 'Instructor',  
4   privileges: [  
5     {
```

```
6     resource: { db: 'edtech_analytics', collection: 'users' },
7     actions: ['find']
8 },
9 {
10     resource: { db: 'edtech_analytics', collection: 'courses' },
11     actions: ['find', 'insert', 'update', 'remove']
12 }
13 ],
14 roles: []
15 });
```

Listing 2: Създаване на потребителски роли и присвояване на привилегии

Този подход гарантира високо ниво на сигурност, предотвратявайки неоторизиран достъп директно на ниво сървър.

3.2.2. CRUD операции

Системата реализира пълния набор от CRUD операции върху трите колекции:

Създаване (Create):

- Добавяне на нов потребител с масив от интереси
- Създаване на курс с вложени модули и лекции
- Регистрация на студент в курс с начални параметри

Четене (Read):

- Филтриране по роля и статус на активност:

```
1 db.users.find({role: "student", is_active: true})
```

- Търсене по регулярен израз в заглавия на курсове:

```
1 db.courses.find({title: {$regex: "Data", $options: "i"}})
```

- Филтриране по диапазон на цени:

```
1 db.courses.find({price: {$gte: 200, $lte: 400}})
```

- Комбинирани условия с OR:

```
1 db.progress.find({$or: [  
2   {status: "completed"},  
3   {status: "in_progress"}  
4 ]})
```

Актуализиране (Update):

- Добавяне на нова оценка в масива на резултатите:

```
1 db.progress.updateOne({_id: ObjectId("...")}, {$push:  
2   {test_results:  
3     {module: 2, score: 92, attempts: 1, date_taken: new Date()  
4   }  
5 })
```

- Увеличаване на текущата лекция:

```
1 db.progress.updateOne({_id: ObjectId("...")}, {$inc:  
2   {current_lecture: 1}  
3 })
```

- Актуализиране на множество документи:

```
1 db.users.updateMany({role: "student"}, {$set: {is_active: true}})
```

Изтриване (Delete):

- Премахване на неактивни профили:

```
1 db.users.deleteMany({is_active: false,  
2   last_login: {$lt: new Date("2024-01-01")}})
```

- Изтриване на конкретен запис по ID:

```
1 db.progress.deleteOne({_id: ObjectId("...")})
```

3.2.3. Агрегиращи операции

За анализ на данни е реализирана агрегираща заявка за намиране на топ 5 най-активни студенти [11]:

```
1 db.progress.aggregate([
2   { $match: { status: 'completed' } },
3   {
4     $group: {
5       _id: '$student_id',
6       completedCourses: { $sum: 1 },
7       averageScore: { $avg: '$test_results.score' },
8       lastActivity: { $max: '$last_activity' }
9     }
10  },
11  { $sort: { completedCourses: -1, averageScore: -1 } },
12  { $limit: 5 },
13  {
14    $lookup: {
15      from: 'users',
16      localField: '_id',
17      foreignField: '_id',
18      as: 'student'
19    }
20  },
21  { $unwind: '$student' },
22  {
23    $project: {
24      studentName: '$student.name',
25      email: '$student.email',
26      completedCourses: 1,
27      averageScore: { $round: ['$averageScore', 1] },
28      lastActivity: 1
29    }
30  }
31 ])
```

Listing 3: Агрегираща заявка за анализ на топ 5 активни студенти

Тази заявка демонстрира използването на операторите `$match`, `$group`, `$sort`, `$limit`, `$lookup` и `$project` за комплексен анализ на данни от две колекции.

3.3. Тестване и обработка на грешки

Системата включва автоматизирани unit тестове, които покриват както успешните CRUD операции, така и сценарии с невалидни заявки за проверка на обработката на грешки. По-долу са представени логове от тестовете за невалидни и неуспешни заявки:

1. Заявка към несъществуващ маршрут (404 Not Found):

```
1 GET /nonexistent
2 Response Status: 404
3 Response Body:
4 {
5   "error": "Route not found"
6 }
```

Listing 4: Лог от заявка към несъществуващ маршрут

2. Заявка с невалиден формат на идентификатора (500 Internal Server Error):

```
1 PUT /users/invalid-id
2 Response Status: 500
3 Response Body:
4 {
5   "error": "input must be a 24 character hex string, 12 byte Uint8Array, or
6   an integer"
7 }
```

Listing 5: Лог от заявка с невалиден ObjectId

3. Заявка за изтриване на несъществуващ потребител (404 Not Found):

```
1 DELETE /users/507f1f77bcf86cd799439011
2 Response Status: 404
3 Response Body:
4 {
5   "message": "User not found"
6 }
```

Listing 6: Лог от заявка за триене на несъществуващ запис

Тези тестове гарантират стабилността на приложението при неправилно подадени входни данни.

Архитектурата на системата осигурява ефективно управление на образователни



данни с висока степен на гъвкавост и производителност, като същевременно поддържа целостта и консистентността на информацията [12].

IV. ЗАКЛЮЧЕНИЕ

В рамките на тази курсова работа беше успешно разработена работеща платформа за персонализирано обучение, базирана на MongoDB като нерелационна база данни. Проектът демонстрира пълната реализация на изискванията от заданието, включително правилно структуриране на данни, изпълнение на CRUD операции, агрегиращи заявки и управление на достъпа чрез роли.

4.1. Обобщение на проекта

В настоящата курсова работа беше разработена цялостна система за персонализирано електронно обучение, базирана на документо-ориентираната NoSQL база данни MongoDB. Проектът успешно демонстрира практическото приложение на нерелационни бази данни при решаване на реални бизнес проблеми в образователната сфера.

Основните постигнати резултати включват:

1. **Реализирана работеща база данни** с три основни колекции (users, courses, progress), съдържаща над 30 реалистични записа
2. **Пълна имплементация на CRUD операции** с разнообразни техники за филтриране, включително регулярни изрази, диапазонни търсения и комбинирани логически условия
3. **Комплексни агрегиращи заявки** използващи Aggregation Framework с оператори като \$group, \$lookup, \$unwind, \$match и \$sort
4. **Механизъм за управление на достъпа** чрез Role-Based Access Control с три дефинирани роли (DataAnalyst, Instructor, Student)
5. **REST API** разработен с Node.js и Express.js, предоставящ програмна интерфейс към базата данни
6. **Интерактивен уеб интерфейс** за визуализация и мониторинг на данните в реално време

7. **Контейнеризация** на приложението с Docker за лесно разполагане

8. **Автоматизирани тестове** и CI/CD pipeline с GitHub Actions

4.2. Основни изводи

Практическата работа с MongoDB потвърди теоретичните предимства на документо-ориентирания модел за образователни приложения:

- **Гъвкавост в моделирането:** Липсата на фиксирана схема позволи естествено представяне на комплексни структури като курсове с вложени модули и лекции
- **Висока производителност:** Aggregation Framework осигури ефективни аналитични операции върху големи обеми данни без необходимост от множество JOIN операции
- **Масщабируемост:** Архитектурата поддържа хоризонтално мащабиране чрез шардинг и репликация
- **Разработческа ефективност:** JSON-подобният формат улесни интеграцията между различни компоненти на системата

Системата успешно демонстрира как NoSQL базите данни могат да се използват за решаване на проблеми, свързани с обработка на йерархични данни в образователни платформи, като същевременно поддържа консистентност и надеждност.

4.3. Постигнати резултати

Основните постижения на проекта включват:

1. **Структура на данни:** Създадени са три добре организирани колекции с баланс между вложени документи и референции, които отразяват естествената йерархия на образователните данни.
2. **CRUD операции:** Реализирани са пълни операции за създаване, четене с различни филтри (регулярни изрази, диапазони, комбинирани условия), актуализиране с оператори като \$set, \$inc, \$push и изтриване.

3. **Агрегиращи заявки:** Разработена е комплексна заявка за анализ на топ активни студенти, използваща pipeline с \$match, \$group, \$sort, \$lookup и \$project.
4. **Управление на достъпа:** Внедрена е система за ролево-базиран контрол с три типа роли (DataAnalyst, Instructor, Student) с различни нива на достъп.
5. **REST API:** Създаден е пълен REST API с Express.js, който предоставя достъп до всички операции.
6. **Уеб интерфейс:** Разработен е интерактивен уеб интерфейс за демонстрация на функционалностите без необходимост от външни инструменти.
7. **Тестване:** Написани са unit тестове с високо покритие, използващи Jest и Supertest.
8. **Девопс:** Конфигурирани са Docker контейнеризация и GitHub Actions за CI/CD [8].

Базата данни съдържа над 30 реалистични записа, разпределени в трите колекции, което позволява адекватно тестване на всички функционалности.

4.4. Предимства на нерелационния подход

Проектът демонстрира ключовите предимства на документо-ориентираните бази данни в сферата на електронното обучение:

- **Гъвкавост:** Лесно адаптиране към променящи се изисквания без миграции на схема.
- **Естествено моделиране:** Вложените документи за модули и лекции отразяват реалната структура на курсовете.
- **Производителност:** Агрегиращите операции позволяват бърз анализ на големи обеми данни.
- **Мащабируемост:** Архитектурата поддържа хоризонтално мащабиране за растящи образователни платформи [13].

4.5. Предизвикателства и решения

В процеса на разработка бяха преодолені няколко технически предизвикателства:

1. **Моделиране на връзките:** Вместо традиционни foreign key constraints беше използвана application-level логика за поддържане на референтна цялост
2. **Производителност на агрегациите:** Оптимизация на aggregation pipeline чрез подходящ ред на операторите и използване на индекси
3. **Безопасност:** Имплементация на RBAC чрез MongoDB's вградени механизми за управление на потребители и роли
4. **Тестируемост:** Разработване на unit тестове за асинхронни операции с база данни използвайки mocking техники

4.6. Препоръки за бъдещо развитие

Въз основа на натрупания опит, се препоръчват следните насоки за разширение на системата:

1. **Внедряване на времеви серии оптимизации:** Използване на MongoDB Time Series Collections за по-ефективно съхранение на данни за активността на студентите
2. **Добавяне на геопространствени индекси:** Реализация на геопространствени заявки за анализ на локации на студенти (ако се събират такива данни)
3. **Интеграция с облачни услуги:** Разширение с MongoDB Atlas за глобално разпределено съхранение и автоматично мащабиране
4. **Машинно обучение:** Имплементация на предиктивна аналитика за прогнозиране на успеха на студентите въз основа на исторически данни
5. **Микросървисна архитектура:** Разделяне на системата на независими услуги за подобряване на мащабируемостта и надеждността
6. **Реално време комуникация:** Добавяне на WebSocket връзки за live notifications и collaborative learning features

4.7. Заключение

Проектът успешно демонстрира потенциала на NoSQL базите данни за разработка на модерни образователни системи. Изборът на MongoDB като технологична основа се оказва правилен за решаване на комплексни проблеми в сферата на електронното обучение.

Работата подчертава важността на правилното моделиране на данни, ефективното използване на aggregation framework и необходимостта от подходящи механизми за сигурност. Получените резултати предоставят солидна основа за по-нататъшно развитие и могат да служат като референтен модел за подобни системи в образователната индустрия.

Технологиите, използвани в проекта (MongoDB, Node.js, Docker, CI/CD), представляват съвременния стандарт за разработка на мащабируеми приложения и предоставят отлични възможности за бъдещо разширение и интеграция.

V. ТЕСТВАНЕ И РЕЗУЛТАТИ

За проверка на коректността на разработената система за персонализирано обучение са реализирани автоматизирани тестове с рамката Jest. Тестовите покриват следните основни аспекти:

- Свързване с базата данни и правилно инициализиране на сесиите.
- Пълна проверка на CRUD операциите за всички основни колекции: Потребители (Users), Курсове (Courses) и Прогрес (Progress).
- Проверка на агрегиращите заявки за генериране на аналитични отчети (Топ студенти, статистики за записвания).
- Валидация на обработката на грешки (невалидни идентификатори, несъществуващи пътища).

По-долу е представен логът от изпълнението на тестовите, който потвърждава, че всички 22 теста преминават успешно:

```
1 > project-1@1.0.0 test
2 > jest
3
4 PASS tests/test-api.js
5   EdTech Analytics API Tests
6     Health Check
7       + GET /health should return OK status
8     Users CRUD
9       + POST /users should create a new user
10      + GET /users should return users
11      + GET /users with filters should work
12      + PUT /users/:id should update a user
13      + DELETE /users/:id should delete a user
14     Courses CRUD
15       + POST /courses should create a new course
16       + GET /courses should return courses
17       + GET /courses with price filter should work
18       + PUT /courses/:id should update a course
19       + DELETE /courses/:id should delete a course
20     Progress CRUD
21       + POST /progress should create a new progress record
```




```
22 + GET /progress should return progress records
23 + GET /progress with status filter should work
24 + PUT /progress/:id should update a progress record
25 + DELETE /progress/:id should delete a progress record
26 Analytics Endpoints
27 + GET /analytics/top-active-students should return analytics data
28 + GET /analytics/students-completed-by-alternative should return data
29 + GET /analytics/course-enrollment-stats should return enrollment
30 statistics
31 Error Handling
32 + GET /nonexistent should return 404
33 + PUT /users/invalid-id should return 500
34 + DELETE /users/nonexistent-id should return 404
35
36 Test Suites: 1 passed, 1 total
37 Tests:      22 passed, 22 total
38 Snapshots:  0 total
39 Time:       0.509 s
40 Ran all test suites.
```

Listing 7: Лог от изпълнение на автоматизирани тестове - Проект 1

ЛИТЕРАТУРА

- [1] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012.
- [2] MongoDB Inc., “Mongodb documentation,” 2024. Accessed: 2024-02-13.
- [3] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [4] E. Brewer, “Cap twelve years later: How the rules have changed,” 2012. Accessed: 2024-02-13.
- [5] M. Inc., “Mongodb documentation.” <https://docs.mongodb.com>, 2024. Accessed: 2024-01-15. <https://docs.mongodb.com>.
- [6] MongoDB Inc., “Mongodb indexing strategies,” 2024. Accessed: 2024-02-13.
- [7] OpenJS Foundation, “Node.js documentation,” 2024. Accessed: 2024-02-13.
- [8] Docker Inc., “Docker documentation,” 2024. Accessed: 2024-02-13.
- [9] K. Banker, P. Bakkum, S. Verch, D. Garrett, and D. Hows, *MongoDB in Action*. Manning Publications, 2011.
- [10] JSON Schema Org, “Json schema validation specification,” 2024. Accessed: 2024-02-13.
- [11] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The Definitive Guide*. O’Reilly Media, 3rd ed., 2019.
- [12] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [13] Microsoft Azure, “Data partitioning guidance,” 2024. Accessed: 2024-02-13.